

MONGODB & DEPLOYMENT GUIDE

Developer Execution Handbook

Your Actionable Next Steps

A step-by-step PDF execution playbook outlining exactly how to establish local or cloud MongoDB instances, connect your application, activate dynamic AI interfaces, and deploy your frontend and backend repositories on Render and Vercel services.

CONFIDENTIAL DEVELOPER ARTIFACT

AI Smart Expense Tracker & Savings Planner v1.0.0
Database Integration & Live Web Setup Blueprint

Actionable Next Steps: MongoDB Configuration & Deployment Guide

This guide outlines the precise tasks you can perform from your side to connect your database (locally or in the cloud), activate live AI budgeting features, and host your full-stack application on the web.

[MongoDB] 1. MongoDB Database Tasks (Your Side)

To store user sessions, transactions, and savings goals, the application requires a running MongoDB database. You have two options: a local database for offline development, or a secure cloud database in MongoDB Atlas.

Option A: Local MongoDB Setup (For Offline Testing)

If you want to run the entire system offline on your local computer, do the following:

- **Step 1: Install MongoDB Community Server**

Download and install the community edition from:

``https://www.mongodb.com/try/download/community``

- **Step 2: Start the Database Service**

- **Windows:** Press ``Win + R``, type ``services.msc``, locate ``MongoDB Server``, and click **Start** or **Restart**.

- **Mac:** Run ``brew services start mongodb-community`` in terminal.

- **Step 3: Install MongoDB Compass (Visualizer)**

Download Compass from: ``https://www.mongodb.com/try/download/compass``. This tool lets you view and edit your ``users``, ``expenses``, and ``savings`` database collections inside a visual screen.

- **Step 4: Check Backend Configuration**

Open ``backend/.env`` and ensure the database connection link points to your localhost:

```
MONGO_URI=mongodb://localhost:27017/expense_tracker
```

Option B: Cloud MongoDB Atlas Setup (Recommended & Required for Deployment)

If you want a secure database hosted in the cloud that your live website can access:

1. **Sign Up for Free:** Create a free account at ``https://www.mongodb.com/cloud/atlas``.

2. **Deploy a Free Database:** Build a free shared database tier called **M0 Sandbox**.

3. **Configure Network Access (Crucial):**

- In the Atlas left sidebar, click **Network Access**.

- Click **Add IP Address**.

- Choose **Allow Access from Anywhere** (IP ``0.0.0.0/0``). This is necessary so that cloud hosting services (like Render) can talk to your database.

4. **Create Database User Credentials:**

- Click **Database Access** in the left sidebar.
- Click **Add New Database User**.
- Choose a Username (e.g. `db\_admin`) and a secure Password. Save these!

5. Get Your Connection String:

- Go to **Database** (clusters overview).
- Click the **Connect** button on your cluster.
- Select **Drivers** (Python).
- Copy the connection string (SRV link) which looks like:

```
`mongodb+srv://db_admin:<password>@cluster0.abcde.mongodb.net/?retryWrites=true&w=majority`
```

6. Update Config:

- Paste this link into your local `backend/.env` file.
- Replace ``<password>`` with the actual database password you created in step 4.

[AI] 2. Activate OpenAI Financial Insights

The system features a dual-engine architecture. If no OpenAI key is set, it falls back to a mathematical rules engine. To activate the fully conversational ChatGPT budgeting coach:

1. **Retrieve API Key:** Log in to your OpenAI dashboard at `https://platform.openai.com/api-keys`.
2. **Generate a Secret Key:** Click **Create new secret key** and copy it.
3. **Add Key to Environment:**

- Open `backend/.env` and paste your key:

```
OPENAI_API_KEY=sk-proj-YOUR_ACTUAL_KEY_HERE
```

- Restart your Flask backend to apply the changes.

[Cloud] 3. Full-Stack Production Deployment Checklist

To put your AI Smart Expense Tracker & Savings Planner online so anyone can log in and manage their finances:

Step 1: Push Local Repository to GitHub

Since all deployment tools sync automatically with GitHub, you need to save your workspace files online:

1. **Create a New Repo:** Go to `https://github.com/new` and create a blank, private repository named `ai-smart-expense-tracker`. Do not initialize it with a README.
2. **Add Remote Origin:** Run this in your workspace terminal (command prompt or PowerShell):

```
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/ai-smart-expense-tracker.git
```

3. **Push Code:** Upload all the local files we initialized into GitHub:

```
git branch -M main
git push -u origin main
```

Step 2: Deploy Flask Backend to Render (Free Cloud Hosting)

1. **Sign Up:** Log in to `https://render.com` using your GitHub account.
2. **Create Web Service:** Click **New +** and select **Web Service**.
3. **Connect Repo:** Import your `ai-smart-expense-tracker` repository.
4. **Set Configuration Parameters:**
 - **Name:** `expense-tracker-backend`
 - **Region:** Select the closest region to you.
 - **Branch:** `main`
 - **Runtime:** `Python`
 - **Build Command:** `pip install -r backend/requirements.txt`
 - **Start Command:** `gunicorn --cwd backend/app app:app` (Gunicorn is standard for production Flask deployments).
5. **Inject Environment Variables:**

Under the **Environment** tab, click **Add Environment Variable** and insert:

 - `MONGO_URI` = `*(Your MongoDB Atlas connection link)*`
 - `JWT_SECRET` = `*(Any long secure random string for JWT signatures)*`
 - `OPENAI_API_KEY` = `*(Your OpenAI Secret Key)*`
 - `PORT` = `10000`
6. **Deploy:** Click **Deploy Web Service**. Once built, Render will assign a public HTTPS link (e.g. `https://expense-tracker-backend.onrender.com`). Copy this URL!

Step 3: Deploy React Frontend to Vercel (Free Frontend Hosting)

1. **Sign Up:** Go to `https://vercel.com` and log in with your GitHub account.
2. **Import Project:** Click **Add New > Project**, and import `ai-smart-expense-tracker`.
3. **Edit Project Settings:**
 - **Framework Preset:** Select **Vite**.
 - **Root Directory:** Set this to `frontend` (click Edit, select the `frontend` folder, and save).
 - **Build Command:** `npm run build`
 - **Output Directory:** `dist`
4. **Inject API Route URL:**

Under **Environment Variables**, add the endpoint that links your frontend to your new Render backend:

 - **Key:** `VITE_API_URL`
 - **Value:** `https://expense-tracker-backend.onrender.com/api` `*(make sure to append /api at the end of your Render link!)*`
5. **Deploy:** Click **Deploy**. Vercel will compile your code and produce a secure, fast URL (e.g. `https://ai-smart-expense-tracker.vercel.app`) to access your app from any browser!